Final Project Report

B2B Intelligent Order & Inquiry Assistant

for **Salva Import**

Student: [Your Name]
Course: [Course Name]
Instructor: [Instructor]
Date: October 22, 2026

Summary. Salva Import sells Cumbé, a premium Colombian aguardiente, to bars, restaurants and shops. This project delivers a Telegram assistant, built with n8n, that lets these B2B clients ask product questions and place restock orders at any hour, in English, French or Spanish. The assistant understands free text with Claude, answers product questions from the brand's official content through a retrieval sub-workflow, checks and reserves live stock through a REST API, sends a priced receipt, and hands the conversation to the sales team when it cannot finish on its own. Every request is logged with a status. The system runs 24/7 on Salva Import's server behind `https://bot.cumbespirits.fr`. It reaches the Level 3 (Advanced) target set in the subject proposal without using the planned cut line, and adds six improvements beyond the original scope.

1. Context and objective

Salva Import's clients are professionals who restock between services, often outside office hours. Before this project, every question ("What does Cumbé taste like?") or order ("3 cases of the 1 L") had to wait for a sales representative to read an email.

The need, in one sentence: Salva Import's B2B clients need a way to instantly ask questions about Cumbé and place restock orders at any hour, instead of waiting for business hours to reach a sales representative.

The product range handled by the assistant is Cumbé Premium Aguardiente (29% ABV, Pimpinella anise, rested in Colombian oak casks) in 70 cl and 1 L bottles, sold by cases of 6.

2. What the assistant does

From the client's point of view, the assistant is a Telegram contact that:

- **understands free text** in English, French and Spanish, and always replies in the client's language;
- **answers product questions** (tasting notes, serving, the four official cocktails) using only Salva Import's own content, and says so when it does not know;
- **takes orders**: it matches the product and bottle size, checks live stock, reserves the bottles and sends a receipt with unit price and total;
- **asks a clarifying question** when an order is incomplete ("70 cl or 1 L?") and remembers the answer for 30 minutes, so "1L" is enough to complete the order;
- **escalates to a human**: if the inventory system is unavailable, the sales team is alerted in a private Telegram group and can chat with the client through the bot.

3. Architecture

3.1 Infrastructure

The solution runs as five Docker services on Salva Import's existing server (OVH VPS, Ubuntu, 4 vCPU, 7.6 GB RAM), next to the company website (Figure 1).

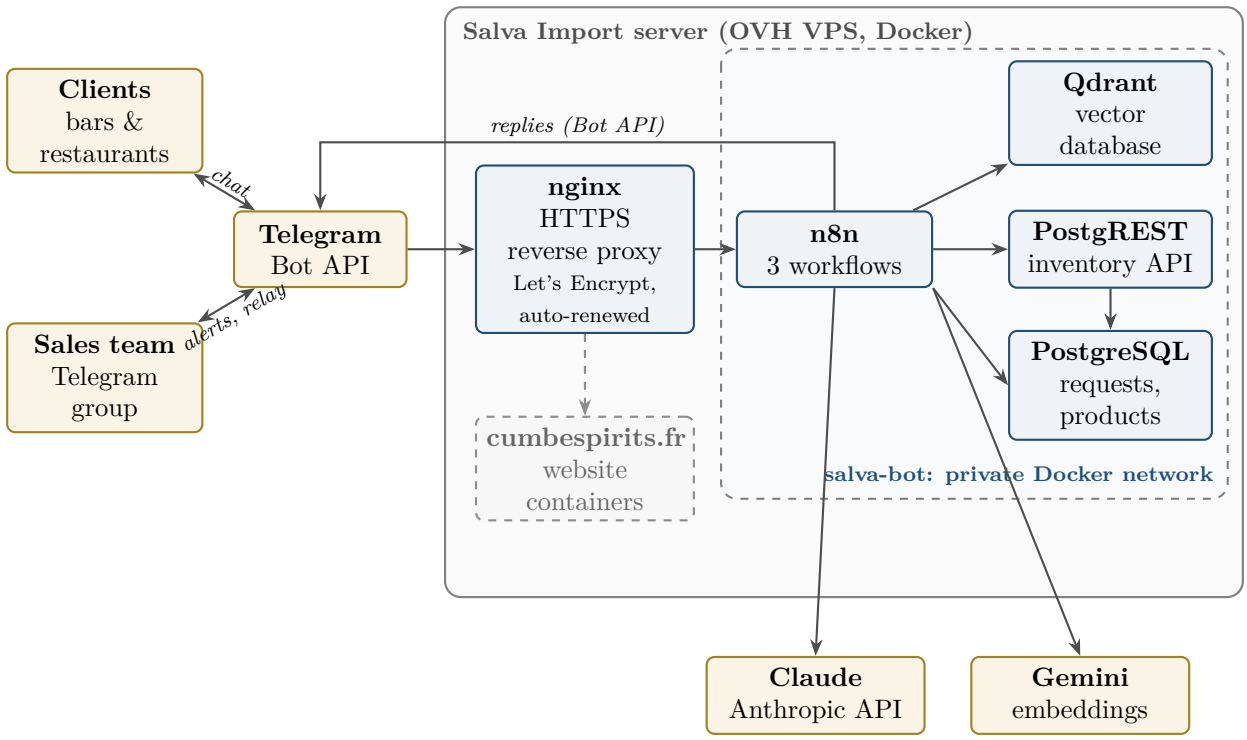


Figure 1: Infrastructure. Only nginx is reachable from the internet; the bot's services share a private Docker network and are never exposed directly.

n8n runs three workflows: *Salva – Main* (the assistant), *Salva – Product knowledge* (the retrieval sub-workflow) and *Salva – Load knowledge* (a one-off loader for the vector database).

PostgreSQL stores every request with its status (`requests`) and the catalogue with prices and stock (`products`).

PostgREST turns the `products` table into a REST API. It connects with a read-only role that can read products and call one stock-reservation function, nothing else.

Qdrant holds the product knowledge as vectors (embeddings computed by Gemini).

nginx is the website's existing gateway. The bot was added as one extra site, `bot.cumbespirits.fr`, without changing the website's configuration.

Claude (Anthropic) reads every message and writes the answers to product questions.

3.2 The main workflow

Figure 2 shows *Salva – Main* grouped into stages. Node names are in italics, as they appear in n8n.

Reception and routing. *Who is writing?* separates private chats (clients) from replies written by the sales team in their group; any other group message is ignored. *Check handover* then looks for a conversation currently handled by a salesperson (less than two hours of silence). If there is one, the message is forwarded to the group instead of being processed by the bot.

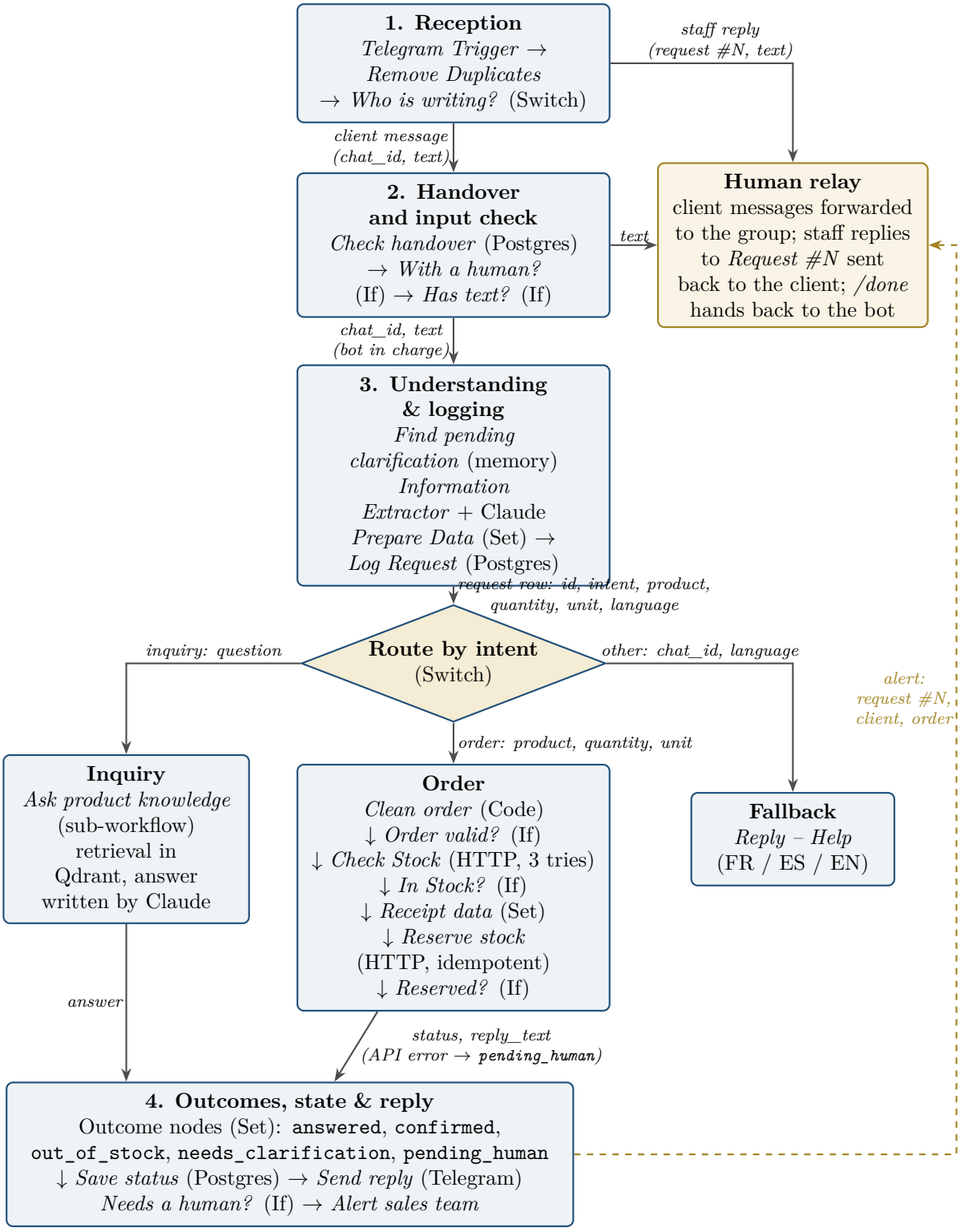


Figure 2: *Salva – Main*, grouped by stage, with the data carried by each arrow. Blue: automated processing; gold: the human relay with the sales team. Messages without text, and any failure of the AI node, go to *Reply – Usage*. Every external call retries 3 times, and any unexpected failure starts *Salva – Error handler*, which posts the workflow name, error and execution URL to the sales group.

Understanding. *Find pending clarification* retrieves a question the bot asked the same client in the last 30 minutes. The *Information Extractor*, driven by Claude, receives the message (or the earlier message, the question and the new answer) and returns structured data: `intent` (`order`, `inquiry` or `other`), `product`, `quantity`, `unit` and `language`. *Prepare Data* flattens this record, and *Log Request* stores it in PostgreSQL *before* any further processing, so no request is ever lost. *Route by intent* has a fallback output: an unexpected classification goes to the help message instead of disappearing.

Inquiry path (sub-workflow and vector database). *Ask product knowledge* calls the sub-workflow *Salva – Product knowledge*, whose only input is the question and whose only output is the answer. Inside, a Question-and-Answer chain retrieves the four closest passages from Qdrant and lets Claude answer *using only those passages*, in the language of the question. The knowledge base contains the official Cumbé content: product facts, tasting notes, serving advice and the four official cocktails (Guaro Sour, Feliz Endings, Pimpinella, Carajillo). Prices and stock are deliberately absent from it: they change, so they stay in the database.

Order path. The AI is good at understanding language; exact business rules are better handled by code. *Clean order* (JavaScript) removes accents and capitals, matches the product to a catalogue SKU, reads the bottle size from expressions such as "1L", "70cl" or "0,7 l", and validates the quantity. If something is missing it writes a short question in the client's language. Otherwise *Check Stock* reads the product from the inventory API, *In Stock?* compares bottles needed with bottles available, *Receipt data* computes the total, and *Reserve stock* deducts the bottles through the API.

Outcomes, state and reply. Each outcome is a Set node that produces the same four fields (`request_id`, `chat_id`, `status`, `reply_text`). This lets a single *Save status* node update the request in the database and a single *Send reply* node answer the client. For example, a confirmed order of 3 cases of Cumbé 1 L produces a receipt for 18 bottles at 153.00 EUR per case, total 459.00 EUR excl. VAT, formatted in the client's language.

Human in the loop. When a request ends in `pending_human`, *Alert sales team* posts it in the private group. A salesperson answers by replying to the alert; the bot delivers the reply to the client and marks the request `handled_by_human`. The client's next messages are relayed to the group until a salesperson replies */done* or two hours pass without activity.

3.3 Data model and request lifecycle

Table / columns	Purpose
`requests`: one row per incoming client message	
`id`, `created_at`, `chat_id`, `client_name`, `message`	who wrote what, and when
`intent`, `product`, `quantity`, `unit`, `language`	what the AI understood
`status`, `bot_reply`	the outcome and what the bot answered
`follow_up_of`	links a short answer to the question it completes
`reserved_sku`, `reserved_bottles`	stock reserved for this request
`human_last_at`	last activity in a conversation handled by a salesperson
`products`: the catalogue	
`sku`, `name`, `category`, `bottle_ml`	product identity and format
`bottles_per_case`, `price_per_case`, `stock_bottles`	ordering unit, price, live stock

Table 1: PostgreSQL tables. `chat_id` is a `BIGINT`: Telegram identifiers exceed the range of a standard integer.

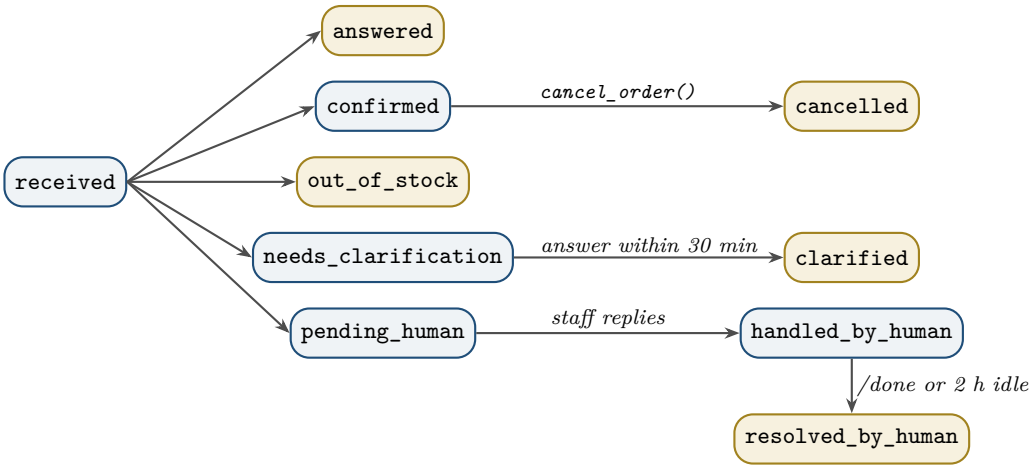


Figure 3: Request statuses. Gold boxes are final states. Greetings and off-topic messages receive the help message and keep the status `received`.

4. Minimum technical requirements

	Requirement	Where it is in the project
✓	Message trigger	*Telegram Trigger*, reached over HTTPS at `bot.cumbespirits.fr`
✓	AI node	*Information Extractor* with Claude (intent, product, quantity, unit, language); Question-and-Answer chain in the sub-workflow
✓	Router / If / Switch	*Who is writing?* and *Route by intent* (Switch, with fallback); *With a human?*, *Order valid?*, *In Stock?*, *Reserved?*, *Needs a human?* (If)
✓	If or Set node	*Prepare Data*, *Receipt data* and the Outcome nodes, which format the receipt and every reply
✓	Code node	*Clean order*: SKU matching, bottle-size parsing, quantity validation, multilingual questions
✓	API connection	*Check Stock* and *Reserve stock*: HTTP calls to the PostgREST inventory API
✓	Database	PostgreSQL (requests with status, catalogue) and Qdrant (vector database)
✓	Sub-workflow	*Salva – Product knowledge*, called by *Ask product knowledge*
✓	Error handling and retries	Retry On Fail (3 attempts, 2 s apart) on every AI, database, HTTP and Telegram node; API errors end in `pending_human` with a polite message and a sales alert; AI errors and messages without text get a usage message; *Salva – Error handler* (Error Trigger) alerts the sales group with the workflow name and execution URL

Table 2: Course requirements and the nodes that implement them.

5. Reliability, safety and security

- **No silent failures.** Both Switch nodes have explicit handling for unexpected input; Postgres look-ups that may return nothing use *Always Output Data*; every external call has retries; API calls and the AI node have an error output; an Error Workflow reports any other failure with a link to the execution.
- **No duplicates.** *Remove Duplicates* drops a Telegram update that was already processed (same `update_id`), so a message delivered twice creates no second row and no second reply.
- **No overselling.** Reservation is a single database operation that only succeeds if enough bottles remain; PostgreSQL locks the product row while it runs:

```
UPDATE products
   SET stock_bottles = stock_bottles - p_bottles
 WHERE sku = p_sku AND p_bottles > 0 AND stock_bottles >= p_bottles
RETURNING stock_bottles;
```

- **Safe retries (idempotency).** The function `reserve_stock` first checks whether the request already holds a reservation. A retried call therefore never deducts the same order twice. `cancel_order` gives the bottles back.
- **Least privilege.** The inventory API cannot modify the catalogue directly and cannot read client data: its role may only read `products` and call `reserve_stock`.
- **Isolation.** The database, the API and Qdrant listen only on the private Docker network or on the server's loopback interface. nginx resolves the bot only when a request arrives, so the website keeps working even if the bot is stopped.
- **Transport and access.** HTTPS with Let's Encrypt certificates renewed and installed au-

tomatically; two-factor authentication on the n8n account; secrets kept in a server-side `.env` file and never exported with the workflows.

6. Deployment and operations

The project was first built on a laptop, then moved to the company server as a Docker Compose project (`salva-bot`). The migration copied the three data volumes (n8n, PostgreSQL, Qdrant), so workflows, encrypted credentials, logged requests and vectors moved unchanged. The tunnel used during development (ngrok) was replaced by an extra site in the website's existing nginx, with its own certificate. During this work, a misconfiguration in the *website's* certificate renewal was found and fixed: it would have caused a certificate error on `cumbespirits.fr` in November.

The project is versioned in a Git repository: Compose file with every image pinned to an exact version and digest, `.env.example` (the real `.env`, which also holds `N8N_ENCRYPTION_KEY`, is git-ignored), database init scripts that rebuild the full schema and the API roles on a fresh volume, the workflows exported as JSON, and a README describing setup from a fresh clone, the environment variables and the tests.

Task	**How**
Check the services	`docker compose ps` in `~/salva-bot`
Follow a run	n8n *Executions* tab, or `docker compose logs n8n`
Back up the database	`pg_dump` through `docker compose exec postgres`
Cancel an order	`SELECT cancel_order(N);` (releases the reserved bottles)
Reserve stock for a manually confirmed order	`SELECT reserve_stock(N, 'SKU', bottles);`
Update the product knowledge	delete the Qdrant collection, then run *Salva – Load knowledge*
Certificates	renewed by certbot; deploy hooks copy them to nginx and reload it

Table 3: Day-to-day operations.

7. Testing and demonstration

#	Client message or action	Expected behaviour
1	"Bonjour"	help message in French (fallback path)
2	"Quel cocktail faire avec du café ?"	the Carajillo recipe, in French (sub-workflow and vector database)
3	"I need 3 cases of Cumbé 1L"	receipt: 18 bottles, 459.00 EUR excl. VAT; stock reduced by 18
4	"Je veux 4 caisses de Cumbé", then "70 cl"	"70 cl or 1 L?", then a receipt for 4 cases of 70 cl (memory)
5	"Quiero 50 cajas de Cumbé 70cl"	out-of-stock answer in Spanish
6	inventory API stopped, then an order	"system busy" message, alert in the sales group, status `pending_human`
7	salesperson replies to the alert; client answers; salesperson sends */done*	two-way conversation through the bot; bot back in charge afterwards
8	the same reservation sent twice	second call returns `already_reserved`; stock unchanged
9	the pinned test message executed twice	second run stops at *Remove Duplicates*: no row, no message
10	a sticker instead of text	usage message in the client's language
11	`bot.cumbespirits.fr/webhook/error-test`	the Error Workflow alerts the sales group with the execution link

Table 4: Test and demonstration scenarios, all run end to end from a phone.

8. From the proposal to the final product

The subject proposal targeted Level 3 (Advanced), with a cut line that would have removed the inventory API and the vector database if time ran short. **The cut line was not needed**: both components are in production. Table 5 lists the six improvements that go beyond the proposal.

Upgrade	Result
Server deployment	runs 24/7 on the company server at `bot.cumbespirits.fr`, with HTTPS
Conversation memory	a short answer ("1L", "6") completes the previous order
Stock reservation	atomic and idempotent; orders can be cancelled
Sales team alerts	`pending_human` requests are posted in a private Telegram group
Two-way relay	salespeople and clients chat through the bot; */done* hands back
Multilingual replies	every message, including clarifying questions, in EN / FR / ES

Table 5: Improvements delivered beyond the subject proposal.

9. Challenges and lessons learned

Symptom	Cause	Lesson
Telegram error “chat not found”	a field in *Fixed* mode sent the expression as plain text	an expression field must show the *fx* badge and a real preview value
“Referenced node doesn’t exist”	node names in `$('...')` are case-sensitive	drag fields from the input panel so n8n writes the exact name
Every order reported “out of stock”	the API’s JSON variant was not recognised, so the reply was kept as plain text	force *Response Format: JSON* on HTTP nodes
Receipt showed “NaN”	a field named `unit price` instead of `unit_price`	consistent field names matter as much as the logic
Bot stopped answering after a test	Telegram allows one webhook per bot; a manual test took it	test the live bot from a phone and follow runs in *Executions*
Workflow stopped without error	a database look-up returned no rows	use *Always Output Data* on optional look-ups
Database started without tables	a missing file was mounted, so Docker created a folder instead	check files exist before mounting them
Telegram IDs rejected	identifiers larger than a 32-bit integer	use `BIGINT`
Ports 80/443 already in use on the server	the website’s nginx owns them	integrate with the existing gateway instead of adding a second one

Table 6: Problems met while building the project, and what they taught.

10. Limitations and next steps

- **Relayed conversations are not stored.** The sales group serves as their history; saving them in the database would allow reporting.
- **Photos and voice messages** are announced to the sales team but not forwarded.
- **The product dictionary** used by *Clean order* duplicates the catalogue; it could be generated from the `products` table so new products need no code change.
- **Free-form product chat** has no memory beyond order clarifications; a full chat memory would allow follow-up questions such as “and the 1 L?”.
- **Personal data.** Client first names, Telegram identifiers and messages are stored; a retention period should be defined and applied (GDPR).
- **Single server.** Backups are manual; scheduled database backups and monitoring of the services would make the system production-grade.